

AFRILANG Stdlib

std/c/simlorenz

Bibliothèque AFRILANG `std/c/simlorenz` (niveau complex, catégorie complex). Elle expose 8 fonction(s) :
loreSum, loreMe

```
`import "std/c/simlorenz" · `use simlorenz`
```

- `loreSum(v list number) ? number`
- `loreMean(v list number) ? number`
- `loreMin(v list number) ? number`
- `loreMax(v list number) ? number`
- `loreVar(v list number) ? number`
- `loreStd(v list number) ? number`
- `loreNorm(v list number) ? list number`
- `simulate(steps number, seed number) ? list number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/simlorenz` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : loreSum, loreMe

```
`import "std/c/simlorenz" . `use simlorenz`  
- `loreSum(v list number) ? number`  
- `loreMean(v list number) ? number`  
- `loreMin(v list number) ? number`  
- `loreMax(v list number) ? number`  
- `loreVar(v list number) ? number`  
- `loreStd(v list number) ? number`  
- `loreNorm(v list number) ? list number`  
- `simulate(steps number, seed number) ? list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/simlorenz"  
use simlorenz
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? loreSum(v list number) ? number  
? loreMean(v list number) ? number  
? loreMin(v list number) ? number  
? loreMax(v list number) ? number  
? loreVar(v list number) ? number  
? loreStd(v list number) ? number  
? loreNorm(v list number) ? list  
? simulate(steps number, seed number) ? list
```

4. Exemples d'utilisation

```
import "std/c/simlorenz"  
use simlorenz  
  
create result = loreSum(/* args */)   
say result  
  
create result = loreMean(/* args */)   
say result  
  
create result = loreMin(/* args */)   
say result  
  
create result = loreMax(/* args */)   
say result  
  
create result = loreVar(/* args */)   
say result  
  
create result = loreStd(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/simlorenz"` en tête de fichier.
? Lancez `afirilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module simlorenz
function loreSum(v list number) returns number
end
function loreMean(v list number) returns number
end
function loreMin(v list number) returns number
end
function loreMax(v list number) returns number
end
function loreVar(v list number) returns number
end
function loreStd(v list number) returns number
end
function loreNorm(v list number) returns list number
end
function simulate(steps number, seed number) returns list number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/simlorenz` (tier complex, category complex). Exports 8 function(s):
loreSum, loreMean, loreMin,

```
`import "std/c/simlorenz" . `use simlorenz`  
- `loreSum(v list number) ? number`  
- `loreMean(v list number) ? number`  
- `loreMin(v list number) ? number`  
- `loreMax(v list number) ? number`  
- `loreVar(v list number) ? number`  
- `loreStd(v list number) ? number`  
- `loreNorm(v list number) ? list number`  
- `simulate(steps number, seed number) ? list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/simlorenz"  
use simlorenz
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? loreSum(v list number) ? number  
? loreMean(v list number) ? number  
? loreMin(v list number) ? number  
? loreMax(v list number) ? number  
? loreVar(v list number) ? number  
? loreStd(v list number) ? number  
? loreNorm(v list number) ? list  
? simulate(steps number, seed number) ? list
```

4. Usage examples

```
import "std/c/simlorenz"  
use simlorenz  
  
create result = loreSum(/* args */)   
say result  
  
create result = loreMean(/* args */)   
say result  
  
create result = loreMin(/* args */)   
say result  
  
create result = loreMax(/* args */)   
say result  
  
create result = loreVar(/* args */)   
say result  
  
create result = loreStd(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/simlorenz"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module simlorenz
function loreSum(v list number) returns number
end
function loreMean(v list number) returns number
end
function loreMin(v list number) returns number
end
function loreMax(v list number) returns number
end
function loreVar(v list number) returns number
end
function loreStd(v list number) returns number
end
function loreNorm(v list number) returns list number
end
function simulate(steps number, seed number) returns list number
end
end
```